Lecture 2. Linear Systems: LU Decomposition

Flops, triangular systems, Gaussian elimination, LU, pivoting, conditioning and stability

Contents

2.1 Algorithms and their cost **2**

2.2 Triangular systems **2**

2.3 Gaussian elimination and the LU decomposition **2**

2.4 Pivoting, conditioning and stability **3**

2.1 Algorithms and their cost

We study algorithms for solving $Ax = b$, a ubiquitous task in statistics. A guiding principle (Gentle):

Mantra

The form of a mathematical expression and the way the expression should be evaluated in practice may be quite different.

An **algorithm** is a finite set of instructions mapping an input to an output. Its cost is counted in **flops**: one floating-point addition, subtraction, multiplication, division or comparison (with the accompanying fetch and store). An algorithm has order $O(f(n))$ if its leading flop count is $c\,f(n)$. For example, Ab with $A \in \mathbb{R}^{m\times n}$ costs $2mn$ flops and AB with $B \in \mathbb{R}^{n\times p}$ costs $2mnp$.

Orders matter enormously at scale: on a machine doing 10^{12} flops per second, a problem of size $n = 10^{12}$ takes about 27 seconds with an $O(n \log n)$ algorithm but about 31700 years with an $O(n^2)$ one. Quicksort and Tukey's fast Fourier transform are celebrated examples that turn $O(n^2)$ into $O(n \log n)$.

2.2 Triangular systems

The strategy for general systems is to reduce them to easy ones. If A is lower triangular, **forward substitution** solves $Ax = b$:

$$x_1 = \frac{b_1}{a_{11}}, \qquad x_i = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j\Big), \quad i = 2, \dots, n,$$

and if A is upper triangular, **back substitution** solves from x_n upwards. Each costs $1 + 3 + 5 + \cdots + (2n-1) = n^2$ flops, and can be done in place (b overwritten by x). In BLAS this is the routine `trsm`.

Useful facts about triangular matrices: the eigenvalues are the diagonal entries; the determinant is their product; products and inverses of (unit) lower triangular matrices are (unit) lower triangular, and similarly for upper triangular. For products, $(AB)_{ij} = \sum_k a_{ik}b_{kj}$ with A, B upper triangular needs $i \le k \le j$, so it vanishes for $i > j$ and equals $a_{ii}b_{ii}$ for $i = j$. For inverses, induct on n with $A = \left(\begin{smallmatrix} A_{11} & a_{12} \\ 0 & a_{22} \end{smallmatrix}\right)$: the bottom-left block B_{21} of $B = A^{-1}$ satisfies $a_{22}B_{21} = 0$, so $B_{21} = 0$ and $A_{11}B_{11} = I$, and $B_{11} = A_{11}^{-1}$ is upper triangular by induction (with unit diagonal if A has one). The lower triangular cases follow by transposition. Consequently an orthogonal triangular matrix Q is diagonal: Q^{-1} is upper triangular while $Q^\top$ is lower triangular, and they are equal.

2.3 Gaussian elimination and the LU decomposition

The **elementary operator matrix** $E_{jk}(c) = I + c\,e_j e_k^\top$ $(j \ne k)$ is unit triangular with $E_{jk}(c)^{-1} = E_{jk}(-c)$, and left-multiplying a matrix by it adds c times row k to row j ($2m$ flops for m columns). Gaussian elimination applies a sequence of such operations to reduce A to upper triangular form.

Combining the operations on column k gives the **Gauss transformation**

$$M_k = E_{nk}(c_{nk}) \cdots E_{k+1,k}(c_{k+1,k}) = I + \mathbf{c}_k e_k^\top, \qquad \mathbf{c}_k = (0, \dots, 0, c_{k+1,k}, \dots, c_{nk})^\top,$$

with $c_{ik} = -a_{ik}^{(k)}/a_{kk}^{(k)}$, which zeros column k below the diagonal. It is unit lower triangular and $M_k^{-1} = I - \mathbf{c}_k e_k^\top$. Elimination computes

$$M_{n-1} \cdots M_1 A = U \quad \text{(upper triangular)}, \qquad \text{so} \qquad A = LU, \quad L = M_1^{-1} \cdots M_{n-1}^{-1}.$$

Proposition 2.1

$L = I - \sum_{k=1}^{n-1} \mathbf{c}_k e_k^\top$: it is unit lower triangular, and its (i,k) entry below the diagonal is the multiplier $\ell_{ik} = a_{ik}^{(k)}/a_{kk}^{(k)}$.

Proof. $M_k^{-1}M_{k+1}^{-1} = (I - \mathbf{c}_k e_k^\top)(I - \mathbf{c}_{k+1}e_{k+1}^\top) = I - \mathbf{c}_k e_k^\top - \mathbf{c}_{k+1}e_{k+1}^\top + \mathbf{c}_k(e_k^\top \mathbf{c}_{k+1})e_{k+1}^\top$, and $e_k^\top \mathbf{c}_{k+1} = 0$ because $\mathbf{c}_{k+1}$ vanishes in positions $\le k+1$. Inductively the product is $I - \sum_k \mathbf{c}_k e_k^\top$. ■

Theorem 2.2 (Existence and uniqueness of LU)

If the leading principal submatrices $A[1{:}k, 1{:}k]$, $k = 1, \dots, n-1$, are nonsingular, then $A = LU$ with L unit lower triangular and U upper triangular. If moreover A is nonsingular, the decomposition is unique and $\det A = \prod_k u_{kk}$.

Proof. *Existence.* The pivot at step k is $a_{kk}^{(k)} = \det A[1{:}k, 1{:}k]/\det A[1{:}k-1, 1{:}k-1]$, because elimination with unit lower triangular operations preserves leading principal minors. So all pivots are nonzero and elimination runs to completion. *Uniqueness.* If $L_1U_1 = L_2U_2$ with A nonsingular, then all factors are invertible and $L_2^{-1}L_1 = U_2U_1^{-1}$. The left side is unit lower triangular and the right side upper triangular, so both equal I. Finally $\det A = \det L \det U = \prod u_{kk}$. (This is how software computes determinants.) ■

For nonsingular A the condition is also necessary, so a nonsingular matrix has an LU decomposition (without pivoting) *if and only if* all its leading principal submatrices are nonsingular. Indeed, partitioning $A = LU$ conformally with the leading $k \times k$ block gives $A[1{:}k, 1{:}k] = L_kU_k$ (the zero blocks of L and U kill the other terms), where L_k, U_k are the leading blocks of L, U. Since $0 \neq \det A = \prod_i u_{ii}$, all $u_{ii} \neq 0$, hence $\det A[1{:}k, 1{:}k] = \prod_{i \le k} u_{ii} \neq 0$.

Cost. Step k updates an $(n-k) \times (n-k)$ block at 2 flops per entry, so LU costs

$$\sum_{k=1}^{n-1} 2(n-k)^2 \approx \tfrac{2}{3}n^3 \text{ flops},$$

done in place (A overwritten by L below the diagonal and U on and above it). Solving $LUx = b$ then costs $2n^2$ (one forward and one back substitution), $\frac{2}{3}n^3 + 2n^2$ in total. With several right-hand sides the factorization is done once.

No inversion. Inverting A via LU solves n systems with right-hand sides $e_1, \dots, e_n$; exploiting the zeros, this costs $\frac{4}{3}n^3$, so $2n^3$ in total, three times the cost of one solve. Hence the rule: *whenever we see $A^{-1}b$, solve $Ax = b$ instead.* Explicit inverses are needed only for standard errors, for very many right-hand sides, or for small n.

2.4 Pivoting, conditioning and stability

For $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $Ax = b$ is trivially solvable but elimination fails at once (zero pivot). Small pivots are just as harmful.

Example 2.1

Solve $0.001x_1 + x_2 = 1$, $x_1 + x_2 = 2$ (exact solution $\approx (1.001, 0.999)$) with two significant digits. Eliminating with the pivot 0.001 gives multiplier 1000 and $(1 - 1000)x_2 = 2 - 1000$, rounded

to $-1000x_2 = -1000$; so $x_2 = 1$ and then $x_1 = 0$. The computed factors $L = \begin{pmatrix} 1 & 0 \\ 1000 & 1 \end{pmatrix}$, $U = \begin{pmatrix} 0.001 & 1 \\ 0 & -1000 \end{pmatrix}$ multiply to $\begin{pmatrix} 0.001 & 1 \\ 1 & 0 \end{pmatrix} \neq A$: the large multiplier wiped out a_{22} and b_2, as in catastrophic cancellation. Swapping the equations first gives multiplier 0.001, $(1 - 0.001)x_2 = 1 - 0.002$, i.e. $x_2 = 1$, $x_1 = 1$: correct to two digits.

Conditioning. The *problem* $A \mapsto A^{-1}b$ is **well conditioned** if small relative perturbations of A cause small relative changes of the solution. It is measured by the **condition number**

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}, \qquad \sigma_{\max}(A) = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}, \quad \sigma_{\min}(A) = \min_{x \neq 0} \frac{\|Ax\|}{\|x\|},$$

which is ∞ for singular A. In Example 2.1, $\kappa(A) = 2.62$: the problem is well conditioned.

Example 2.2 (An ill-conditioned matrix)

The symmetric matrix

$$A = \begin{pmatrix} 5 & 7 & 6 & 5 \\ 7 & 10 & 8 & 7 \\ 6 & 8 & 10 & 9 \\ 5 & 7 & 9 & 10 \end{pmatrix}, \qquad A^{-1} = \begin{pmatrix} 68 & -41 & -17 & 10 \\ -41 & 25 & 10 & -6 \\ -17 & 10 & 5 & -3 \\ 10 & -6 & -3 & 2 \end{pmatrix},$$

looks harmless. Change a_{33} from 10 to 9.99, a relative change of 10^{-3} in one entry. The inverse becomes (to two decimals) $\begin{pmatrix} 71.04 & -42.79 & -17.89 & 10.54 \\ -42.79 & 26.05 & 10.53 & -6.32 \\ -17.89 & 10.53 & 5.26 & -3.16 \\ 10.54 & -6.32 & -3.16 & 2.09 \end{pmatrix}$, whose entries moved by up to 5.4%. Measuring condition by the ratio of maximal relative entrywise changes, the perturbation was amplified about 54 times, so $\kappa(A)$ is at least of that order. The amplification depends on the direction of the perturbation; the worst case is much larger: $\kappa_2(A) = \sigma_{\max}/\sigma_{\min} \approx 2984$, so about 3.5 decimal digits can be lost in solving $Ax = b$.

Stability. An *algorithm* $\tilde{f}$ is **stable** if for every input $\tilde{A}$ within relative distance ϵ of A, the computed $\hat{x} = \tilde{f}(A)$ is within relative distance $O(\epsilon)$ of $\tilde{A}^{-1}b$. Example 2.1 shows that elimination without pivoting is unstable: a well-conditioned problem gives relative error $\|x - \hat{x}\|/\|x\| \geq 1/\sqrt{2}$.

Partial pivoting. Before eliminating column k, swap into row k the row with the largest $|a_{ik}|$, $i \geq k$. This costs only $O(n^2)$ comparisons and gives

$$PA = LU, \qquad P \text{ a permutation matrix}, \quad |\ell_{ij}| \leq 1.$$

Complete pivoting also swaps columns, moving the largest entry of the remaining submatrix to the pivot position: $PAQ = LU$. It is more stable but costs more. Gaussian elimination with partial pivoting (GEPP) is the standard method for general dense systems and is stable in practice, partly because the multipliers are bounded by 1.

Software. Dense linear algebra rests on BLAS (basic operations) and LAPACK (decompositions). The LAPACK routine `getrf` computes $PA = LU$ in place; R's `solve` calls `dgesv`, which factors and solves.